

EventStoryLine 项目结构费曼版

用厨房、档案馆和考试卷把这个仓库讲明白

yifeng_study

2026 年 5 月 7 日



图 1: 费曼式第一句话: 这个项目像一间故事线厨房, 语料是食材, 脚本是厨师, 评测分数是试吃结果。

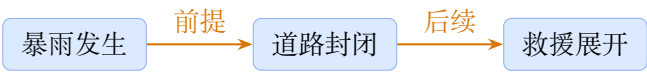
目录

1 先用一句大白话讲清楚	3
2 把仓库想成档案馆	3
3 XML 标注到底是什么	4
4 事件和时间关系怎么标注	5
5 共指关系: 不同说法, 指同一件事	7
6 三种数据版本像三本教材	8

目录	2
7 标准答案是怎么做出来的	8
8 基线脚本像学生做题	9
9 评测脚本像老师批卷	10
10 建议你按这个顺序读	10
11 一张总复习图	11
12 打开方式	11

1 先用一句大白话讲清楚

这个仓库在做一件事：**把新闻里的事件找出来，再判断这些事件之间有没有“故事线关系”。**
比如一篇新闻里出现了这些事件：



项目里的数据和脚本，就是在回答这些问题：

- 哪些词是事件？
- 两个看起来不同的事件提及，是不是其实指同一件事？
- 哪些事件之间有“前因后果”或“情节推进”关系？
- 一个系统猜出的答案，和标准答案比，准不准？

2 把仓库想成档案馆



图 2: 目录不是随机堆放的：它更像一个档案馆，每个房间放一种材料。

真实目录或文件	费曼比喻	你该怎么理解
ECB+_LREC2014/	原始馆藏	原始 ECB+ 语料，像还没加工的新闻档案。
annotated_data/	贴好便签的档案	ESC 标注数据，里面标了事件和情节关系。

evaluation_format/	考试标准答案柜台	已经整理成评测脚本容易读取的格式。
ecb+_naf/	语言学工具卡片	提供 lemma、句子等信息，PPMI 基线会用。
ppmi_events_ecb+_test/	实验材料盒	PPMI 基线相关的测试或中间材料。
create_gold_document.py	档案整理员	把原始标注整理成标准答案文件。
baseline_*.py	答题学生	用规则猜事件关系，生成系统答案。
eval_script.py	批卷老师	对照标准答案，计算精确率、召回率、F1。

3 XML 标注到底是什么

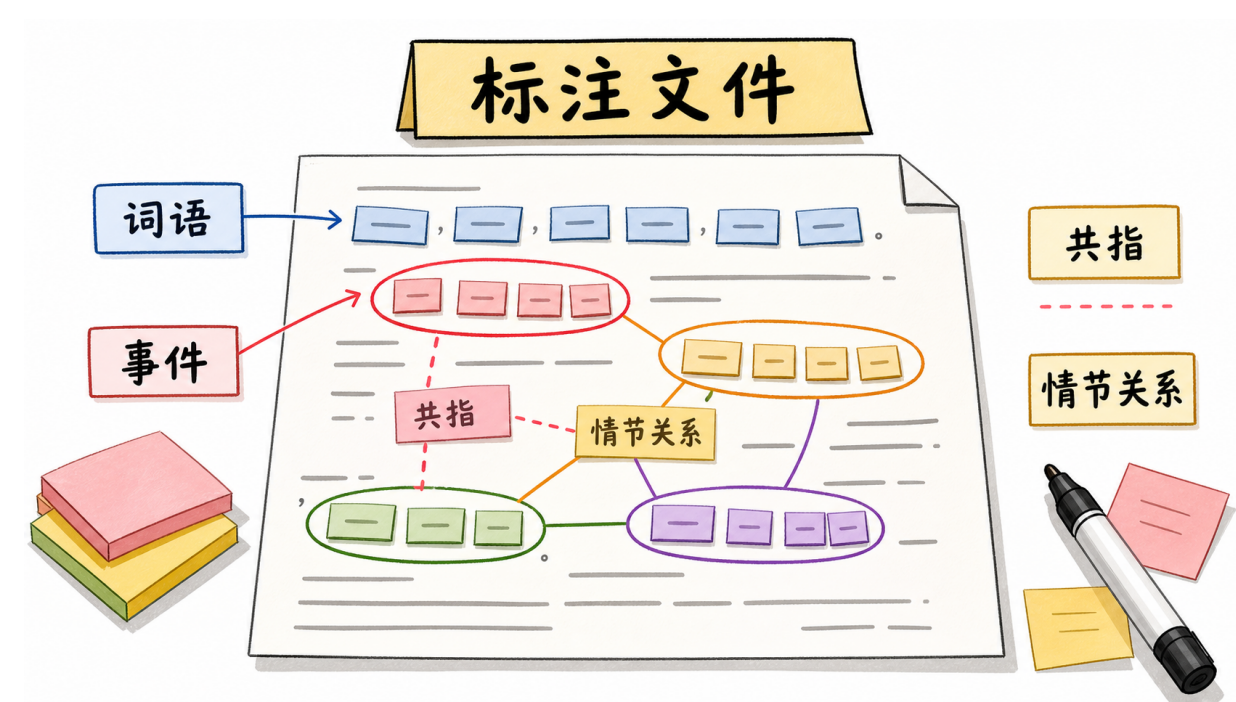


图 3: XML 标注可以理解成“贴满便签的新闻稿”：哪些词是事件，哪些事件互相有关，都写在便签里。

不用先害怕 XML。你可以把一个 XML 文件想成一张新闻稿，旁边贴了很多便签：

- **token**：新闻稿里的每个词，像一个小卡片。
- **Markables**：哪些词被圈出来了，比如“爆炸”“逮捕”“宣布”。
- **Relations**：被圈出来的东西之间有什么关系。
- **PLOT_LINK**：故事线关系，比如一个事件是另一个事件的前提或后续。

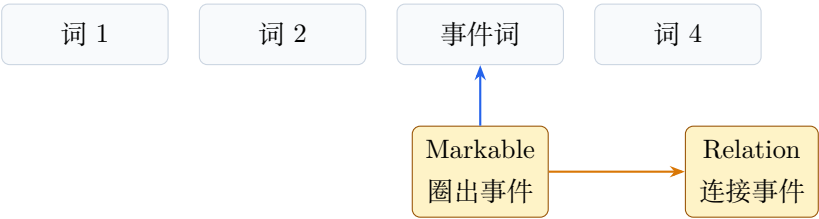


图 4: XML 的关键不是尖括号，而是“词、事件标注、关系标注”三层。

4 事件和时间关系怎么标注

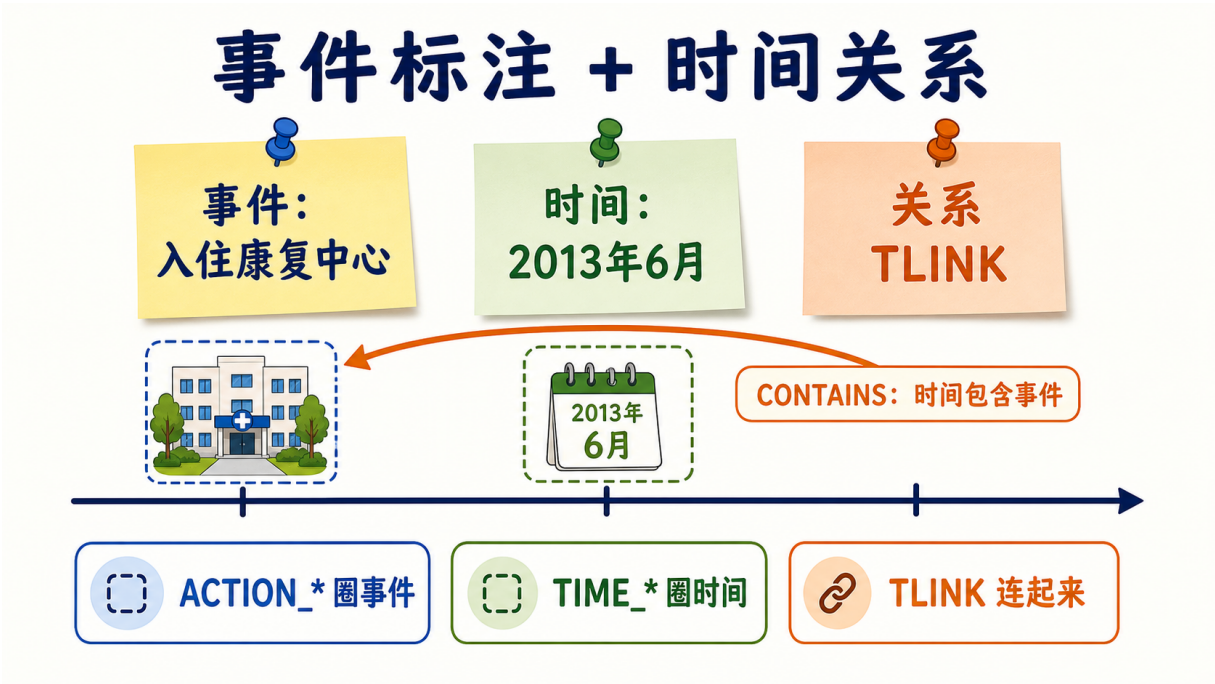
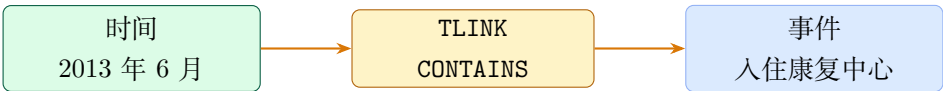


图 5: 事件标注像圈出动作便签，时间标注像圈出日历便签，TLINK 像把二者接到时间轴上的线。

可以把时间关系理解成“新闻稿旁边多贴了一条时间轴”。标注时通常分三步：

1. **先圈事件**：用 ACTION_* 这类标签标出动作或状态，比如“入住”“离开”“宣布”“被捕”。
2. **再圈时间**：用 TIME_* 这类标签标出日期、时间段、一天中的时间，比如“2013 年 6 月”“昨天”“90 天”。
3. **最后连关系**：用 TLINK 说明事件和时间，或者两个事件/两个时间之间的时间关系。

费曼式地说：事件是剧情卡片，时间是日历卡片，TLINK 是把卡片钉到时间轴上的图钉线。



在真实 XML 里，事件和时间都放在 Markables 下面。比如一个事件可能长这样：

```
<ACTION_OCCURRENCE m_id="10" climaxEvent="TRUE" comment="">
  <token_anchor t_id="67"/>
</ACTION_OCCURRENCE>
```

这里的意思是: m_id="10" 是这个事件便签的编号, token_anchor t_id="67" 表示这个事件对应正文里的第 67 个 token。

时间也类似, 只是标签换成 TIME_*:

```
<TIME_DATE m_id="43" DCT="TRUE" value="2010-09-10"/>
```

这里的 value="2010-09-10" 是标准化后的日期。你可以把它理解成: 原文可能写成 “Friday” 或 “Sept. 10”, 标注里尽量归一成机器更好读的时间值。

真正把事件和时间连起来的是 Relations 里的 TLINK:

```
<TLINK r_id="244387" relType="CONTAINS">
  <source m_id="43"/>
  <target m_id="9"/>
</TLINK>
```

这个例子可以读成: m_id=43 这个时间包含了 m_id=9 这个事件。也就是说, 事件 9 发生在时间 43 这个时间范围里。

常见的 TLINK 类型可以先这样记:

关系类型	大白话	例子
CONTAINS	某个时间范围装着某个事件	“2013 年 6 月” 包含 “入住康复中心”。
BEFORE	前者早于后者	“车祸” 早于 “法庭判决”。
AFTER	前者晚于后者	“今天” 晚于 “昨天的治疗”。
OVERLAP	两者时间重叠	“治疗期间” 和 “住院期间” 有重合。
BEGUN_ON	事件从某时间开始	“治疗” 开始于某一天。
ENDED_ON	事件在某时间结束	“治疗” 结束于某一天。

这里要特别分清: TLINK 和 PLOT_LINK 不是一回事。TLINK 主要回答 “什么时候、先后如何”; PLOT_LINK 回答 “故事线里两个事件是什么剧情关系”。所以时间关系更像日历, 故事线关系更像剧情大纲。

5 共指关系：不同说法，指同一件事



图 6: 共指关系像把几张新闻剪报用线连到同一张事件卡片上：说法不同，但指向同一件事。

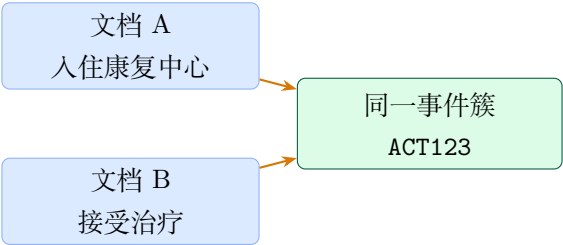
共指这个词听起来很学术，其实可以先理解成一句话：

两个地方写得不一样，但它们说的是同一件事。

比如同一个主题下有两篇相关新闻：

- 文档 A 写：Lohan 入住康复中心。
- 文档 B 写：她接受治疗。

这两个短语表面上不一样，但如果标注者认为它们说的是同一个现实事件，就会把它们放进同一个事件簇里。你可以把这个事件簇想成档案馆里的一张总卡片：



在这个项目里，尤其要分清两层：

- 同文档共指：同一篇新闻里，“爆炸”“这次袭击”“事件”可能指同一件事。
- 跨文档共指：不同新闻文件里，各自写到的事件提及，可能也指同一件现实事件。

ECB+ 支持跨文档共指。它不是在一个 XML 里直接写“去另一个 XML 文件找某个 `m_id`”，而是用共享的事件簇编号或实例编号把它们归到一起。费曼式地说：每篇新闻里的事件提及是小便签，共指链是档案馆给同一事件发的一张总身份证。

这也是为什么 `create_gold_document.py` 需要同时读取 ECB+ 和 ESC: ESC 给出 `PLOT_LINK`, ECB+ 给出共指链。脚本再利用共指链，把“一个事件之间的故事线关系”扩展到更多等价的事件提及对上。

6 三种数据版本像三本教材

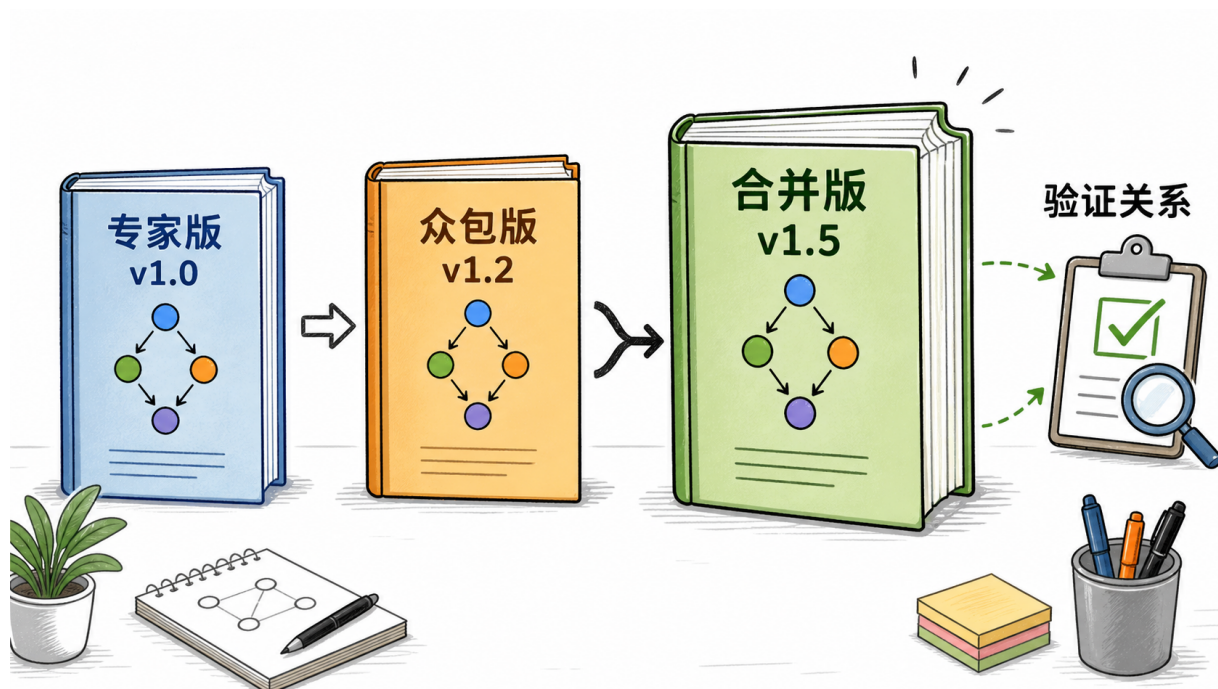


图 7: v1.0、v1.2、v1.5 可以先理解成三本不同来源的故事线教材。

- v1.0: 专家版。可以理解成老师亲自批注的版本。
- v1.2: 众包版。可以理解成很多同学一起标注后汇总的版本。
- v1.5: 合并版。把专家和众包信息结合起来，README 里提到还带有 `origin` 和 `validated` 这样的属性。

如果你只是学习项目结构，建议先看 v1.5，因为它更完整。

7 标准答案是怎么做出来的

`create_gold_document.py` 像一个档案整理员。它同时打开两类档案：

- ECB+ 原始文件：告诉你事件 `mention` 和共指链。
- ESC 标注文件：告诉你事件之间的 `PLOT_LINK`。

然后它做一件很重要的事：**把一个事件关系扩展到共指链里的更多事件提及对。**

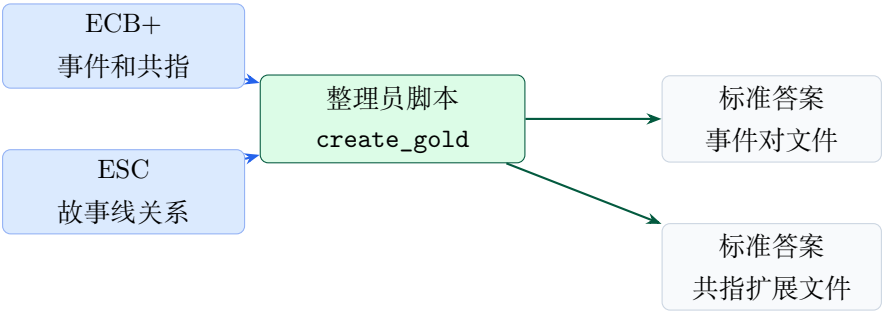


图 8: 标准答案不是凭空来的，而是由 ECB+ 的共指信息和 ESC 的故事线标注合并出来的。

8 基线脚本像学生做题

基线系统不是神经网络模型。它更像学生用一套简单规则做题：

- `baseline_OP.py`: 看到同一句里有多个事件，就把它们两两连起来。
- `baseline_PPMI1.py`: 看事件词的共现强度，觉得常一起出现的事件更可能有关系。
- `baseline_PPMI_CONTAINS.py`: 在 PPMI 的基础上，又看这些事件是不是被同一个时间锚装在一起。

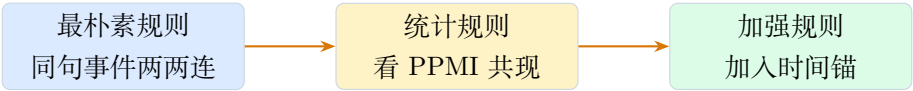


图 9: 三类基线可以按“简单规则、统计规则、时间增强规则”来理解。

9 评测脚本像老师批卷



图 10: 系统输出像学生答案，标准答案像参考答案，评测脚本就是批卷老师。

`eval_script.py` 做的事情很朴素：

1. 读标准答案文件。
2. 读系统输出文件。
3. 看系统猜对了多少、猜错了多少、漏掉了多少。
4. 算精确率、召回率、F1。

精确率
猜出来的有多少是真的

召回率
真的里面找回了多少

F1
两者的折中分

10 建议你按这个顺序读

1. 先看评测文件：打开 `evaluation_format/test_corpus/v1.5/event_mentions_extended/`，感受一行三列长什么样。
2. 再看批卷老师：读 `eval_script.py`，因为它短，而且会告诉你“系统答案应该长什么样”。
3. 再看最简单学生：读 `baseline_0P.py`，理解同句事件配对。
4. 再看档案整理员：读 `create_gold_document.py`，理解标准答案怎样生成。
5. 最后看高级学生：读 `baseline_PPMI1.py` 和 `baseline_PPMI_CONTAINS.py`。

11 一张总复习图

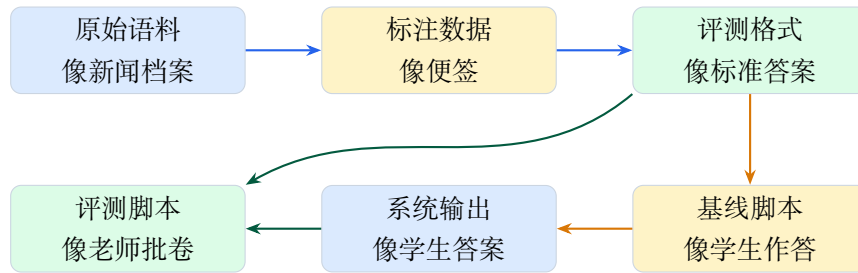


图 11: 记住这条链: 原始语料到标注, 标注到标准答案, 基线生成答案, 评测脚本批改。

12 打开方式

在 VS Code 中打开整个项目:

```
code /Users/luyifeng/Documents/GitHub/EventStoryLine
```

这份费曼版文档在:

```
yifeng_study/feynman_project_guide.tex  
yifeng_study/assets_v2/
```

中文 LaTeX 建议用 XeLaTeX 编译:

```
cd /Users/luyifeng/Documents/GitHub/EventStoryLine/yifeng_study  
xelatex feynman_project_guide.tex
```